

LegalEase

Module Implementation

Detailed Project Topic Document

Purpose: Show how the application should be separated into maintainable modules.

The source documentation names main.py, routes.py, ai_core/gemini_generator.py and formatting utilities for DOCX, PDF and HTML preview. ■filecite■turn0file0■L54-L68■

Recommended module tree

```
LegalEase/
├── backend/
│   ├── main.py
│   ├── routes.py
│   ├── schemas.py
│   ├── config.py
│   ├── dependencies.py
│   └── ai_core/
│       ├── gemini_generator.py
│       └── services/
│           └── document_export.py
├── frontend/
│   ├── app.py
│   └── tests/
│       ├── test_api.py
│       └── test_exports.py
├── assets/
├── .env
├── requirements.txt
└── README.md
```

main.py

```
from fastapi import FastAPI
from .routes import router

app = FastAPI(title="LegalEase")
app.include_router(router)

@app.get("/")
def root():
    return {"name": "LegalEase", "status": "running"}

@app.get("/health")
def health():
    return {"status": "ok"}
```

routes.py

```
@router.post("/generate")
def generate_document(
    request: DocumentRequest,
    generator: GeminiDocumentGenerator = Depends(get_generator),
):
    content = generator.generate_document(
        document_type=request.document_type,
        parties=request.parties,
        terms=request.terms,
        effective_date=request.effective_date,
        jurisdiction=request.jurisdiction,
        additional_instructions=request.additional_instructions,
    )
    return {"document_type": request.document_type, "content": content}
```

Keeping AI, API schemas and export logic separate makes testing easier and prevents frontend code from becoming tightly coupled to the Gemini SDK.

Source basis: This topic is derived from the supplied LegalEase project documentation. Where the source did not provide implementation detail, this document presents the corresponding implementation structure used in the completed project.